Network Pruning :->

Nets are over-parameterised - tons of near zero weights. Remove them -> smaller + faster

Pipeline :->

Pre-Train large-net -> Evaluate Importance -> Prune -> Fine-tune recover accuracy

↓

Small net

->Weight Importance = |w|. near zero -> Prune
->Neuron Importance = how often it fired ≠ 0
Never fires = dead

* Never prune too much at once.

Weight Pruning	Neuron Pruning. ~ Preefered
-> Remove single weights (sparse matrix)	Remove whole neurons (Dense matrix)
-> Hard to speed up	Real GPU speed up

2013

** Lottery Ticket hypothesis (Research Paper)
-> Inside a large randomly Initialised neural network, there exists a small sparse subnetwork that can achieve nearly the same accuracy when trained from its original Initialization.

M T W T F S S

Page No.: YOUVA

Date:

Big Random network

↓ Train

learned Dense network's model

↓ Prune small |w|

Sparse subnetwork

↓ reset to original init

Train again

↓

Matches full accuracy.

** Rethinking LTH (Research Paper 2019)

-> Sparse architecture matters more than the exact weights.

Idea: Keep connectivity mask (Architecture)
Set |w| = 1 (Keep signs of weights but no weight training)

Still decent accuracy

Parameter Quantization :->

Idea:

Train Network in 32-bit Floating point (FP32)
↓↓
store/use fewer bits during Inference.

Instead of FP32
use: 8-bit, 4-bit, 1-bit

less memory + faster hardware (Unsloth, Sandbyte, etc)

↗ Used in NVIDIA TensorRT

M T W T F S S
Page No.: YOUVA
Date:

* FP 32 → INT 8 Memory ↓↓
Speed ↑

$w_q = \text{round}\left(\frac{w}{\Delta}\right) \cdot \Delta, \quad \Delta = \frac{w_{max} - w_{min}}{2^b - 1}$

Δ = quantization of step size
b = no. of bits.

** Weight clustering.
Instead of storing every weight separetly
Group similar weights into K clusters.

Store only cluster centers (centroids)
Index of each weight.

** Huffman coding.
-> Apply clustering
Then Apply compression on its top.

It works well because weight distributions are often Gaussian-like.

(Research Paper - 2016 (HAW))
↓
Results: 35x-49x compression on AlexNet with tiny accuracy loss.

—— x ——

M T W T F S S

Page No.: YOUVA

Date:

EFFicient Architecture Design :

* Depthwise Separable Convolution (DSC)
->split a standard convolutinto into two cheaper operations :

Input : $H \times W \times C_{in}$

① Depthwise Convolution
Apply one $K \times K$ filtor per input channel
->learns spatial features

Flops : $H \cdot W \cdot K^2 \cdot C_{in}$

② Pointwise Convolutior
Apply 1×1 convolution to mix channels.

Flops : $H \cdot W \cdot C_{in} \cdot C_{out}$

Output. $H \times W \times C_{out}$ → Pepthwise (Psc)

↳ whereas Standard one : $H \times W \times C_{out}$ $(C_{in} + K^2)$

↳ $H \times W \times K^2 \times C_{out} \times C_{in}$

Speed up ratio : $\frac{1}{C_{out}} + \frac{1}{K^2}$

Key for Edge Computing, Iot devices.

Uses : MobileNet, Xception

M T W T F S S

Page No.: YOUVA

Date:

* Low Rank Approximation : -

→ Approximate large Matrix
using two smaller Matrices.

$W_{m \times n} \rightarrow W_{m \times n} \approx U_{m \times k} \cdot V_{k \times n}$

$k << \min(m, n)$

Parameter count : $mn \rightarrow k(m+n)$
change

LORA : (Relation)

LORA uses this low-rank matrix decomposition for efficient fine-tuning of LLMs.

Dynamic Computation :->

-> It adapts model cost based on Input difficulty or device capability.

Imp : -> edge AI
-> IoT

* Dynamic Depth
-> Attach small classifiers after the Intermediate layers.

M T W T F S S

Page No.: YOUVA

Date:

Easy Input -> exit early

Hard Input -> use deeper layers.

Pros: -> low latency

-> less energy consumption

* Dynamic width.

-> One Model supports multiple width:

(Runtime chooses width based on available resources)

* Sample - Difficulty Routing

-> Model learns to skip unnecessary layers or residual blocks for easy Inputs.

Knowledge distillation (KD)

Problem: -> Hard labels like [0,0,1,0 ...]

-> only tell the correct class

-> They lose the similarity Information

ex: Prediction of digit 1

1 -> 0.7, 7 -> 0.2, 9 -> 0.1

This shows 1 is somewhat similar to the digit 7.

Teacher -> Student setup.

Teacher Model :-> large /ensemble model

-> High accuracy

-> slow inference

Student model :-> small model

-> faster inference

-> learns from teacher outputs.

loss function.

$$Loss = \alpha \cdot L_{CE}(hard) + (1-\alpha) \cdot T^2 \, KL(soft)$$

T = Temperature

L_{CE} = Cross entropy loss with hard labels

KL = K.L. divergence between teacher & student soft outputs.

α = 0.1 - 0.5

Temp. scaling.

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

T=1 -> normal Softmax (sharp)
T=4-10 -> softer Probablities
-> more learning Information.

Types of Knowledge

1. Response-Based

Where -> final softmax output
Transferred via -> KL/soft cross-entropy

Problem: It treats the model as black box, it only cares about the final output.

-> For deep networks, relying on final output is not enough to train the student's lower layers.

Solution: Enable student to mimic the Internal representations & structural mechanics of teacher.

2. Feature - Based

Where) Hidder layer Activations

Transferred) MSE on feature maps.
via

Concept: Instead of matching Just final output,
= The student is Penalised if its intermediate layers don't match Teacher's

Mechanism: -> Choose a specific layer to match
-> Since Teacher's hidder dimension (D_T) is larger than students (D_S)
we cannot compute MSE directly between Them.

Fix: Introduce a linear Transformation matrix(w)
= to project student's feature into the Teacher's dimensions, then calculate loss

$$MSE\ (Teacher_{features}, Transform(Student_{features}))$$

3. Relation - Based.

where) Sample - pair distances
Transferred) Pairwise distance matching via!

-> It teaches the student relationship between different data points or layers.

Mech1: -> Pass batch of Data through both models
-> Calculate correlation matrix showing how every image relates to other in teacher's embedding space
-> Force student's correlation matrix to match the teacher's.

* IMP
Mech2. (Attention Distillation)
-> for models like TinyBert, DistillBert.

-> In this Distillation the loss function penalisez the Students if its attention heatmaps dont closely mirror's teacher's.

-> This forces the student model to learn exact Syntatic & semantic dependencies.

— X —